NAND Gate Implementation on the Zynq-7000 (ZedBoard)

1. Objective

Implement a NAND gate on a Zynq-7000 ZedBoard (xc7z020clg484-1) by realizing AND in Programmable Logic (PL) and performing the inversion in a bare-metal C program on the Processing System (PS / ARM Cortex-A9). This follows the reference NOR-gate procedure (OR in hardware + NOT in software), adapted to AND + NOT → NAND.

The flow was completed through synthesis, implementation, bitstream generation, hardware export, and C application build. No physical ZedBoard was available, so on-board programming/testing was not done (Section 6).

2. System Architecture

- PL (fabric): a Verilog module computes input_1 AND input_2 → nand_out.
- PS (processor): C code on ps7_cortex9_0 reads nand_out via AXI GPIO, inverts it, and drives the NAND result via a second AXI GPIO.

The two domains connect over AXI4-Lite through an AXI Interconnect, with a Processor System Reset block. The Vivado block design:

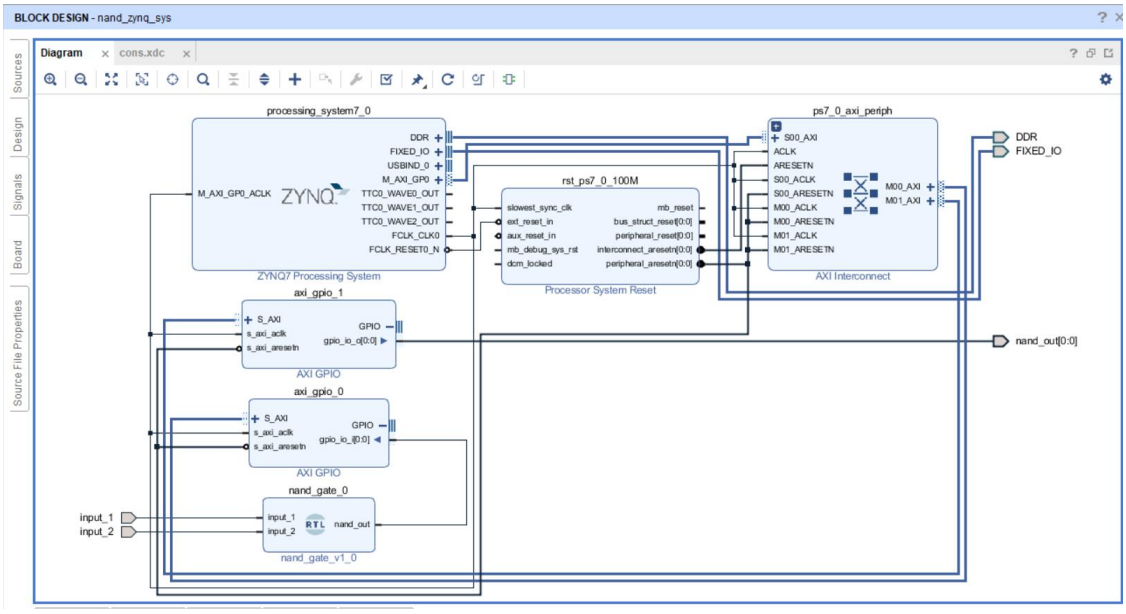

Figure 1 — Vivado block design (nand_zynq_sys): ZYNQ7 Processing System, AXI Interconnect, two AXI GPIO peripherals, and the custom AND-logic RTL module (nand_gate_0).

2.1 Signal Flow

Block / Signal	Role
input_1, input_2	AND inputs (SW0 / SW1).
nand_gate_0	RTL block: nand_out = input_1 & input_2.

Block / Signal	Role
axi_gpio_0 (input)	Brings nand_out into the AXI domain for the PS to read.
processing_system7_0	ARM Cortex-A9; runs the software inversion.
axi_gpio_1 (output)	Carries the inverted result back to native logic.
nand_out (top-level port)	Final NAND result, driven to LED LD0.

3. Hardware Design (Programmable Logic)

The AND function as a single combinational Verilog module (nand_gate.v):

```
module nand_gate(
    input input_1,
    input input_2,
    output reg nand_out
    );
    always @(*)
        nand_out = (input_1 & input_2);
endmodule
```

Note: the file is named nand_gate.v, but it implements pure AND (not inverted NAND) — correct per the assignment, since inversion is deferred to software. Wiring is correct end-to-end; only the filename is a minor misnomer.

3.1 Elaborated Schematic

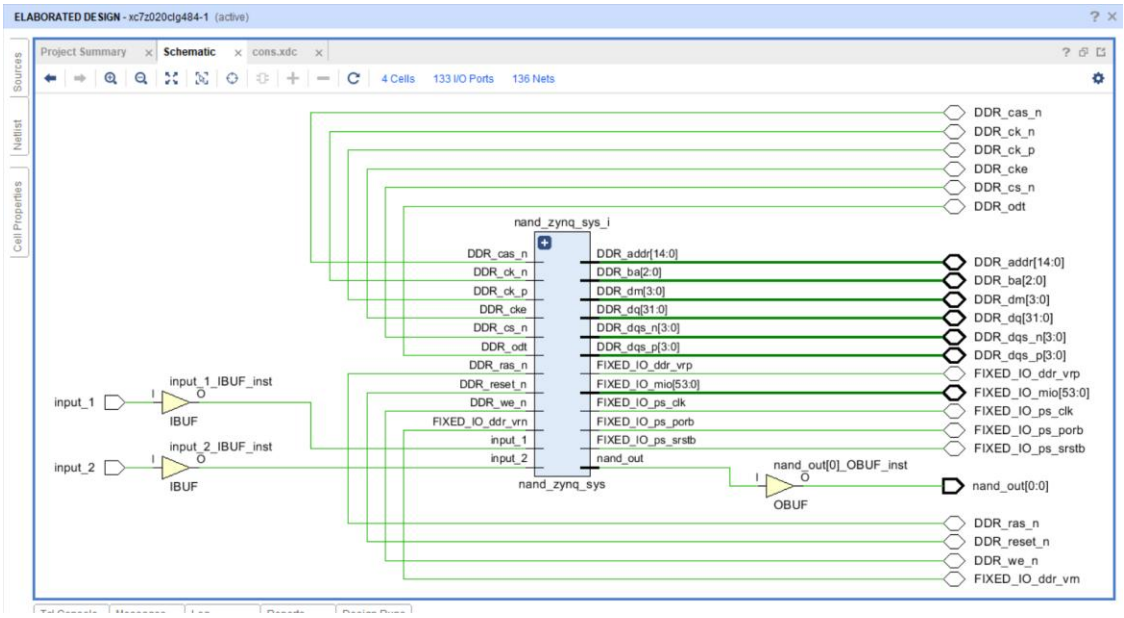

Figure 2 — Elaborated schematic: input_1 / input_2 through input buffers, nand_out through an output buffer to the top-level port.

3.2 Pin Constraints (cons.xdc)

Inputs mapped to ZedBoard switches, output mapped to an LED, so the raw AND signal can be checked independently of the processor:

```
set_property PACKAGE_PIN F22 [get_ports {input_1}];  # "SW0"
set_property PACKAGE_PIN G22 [get_ports {input_2}];  # "SW1"
set_property PACKAGE_PIN T22 [get_ports {nand_out}]; # "LD0"

set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];
```

Based on the standard Avnet ZedBoard master XDC, with only the relevant switch/LED lines active and the required bank-wide IOSTANDARD statements.

3.3 Implementation Results

Placed and routed successfully on the xc7z020clg484-1 device:

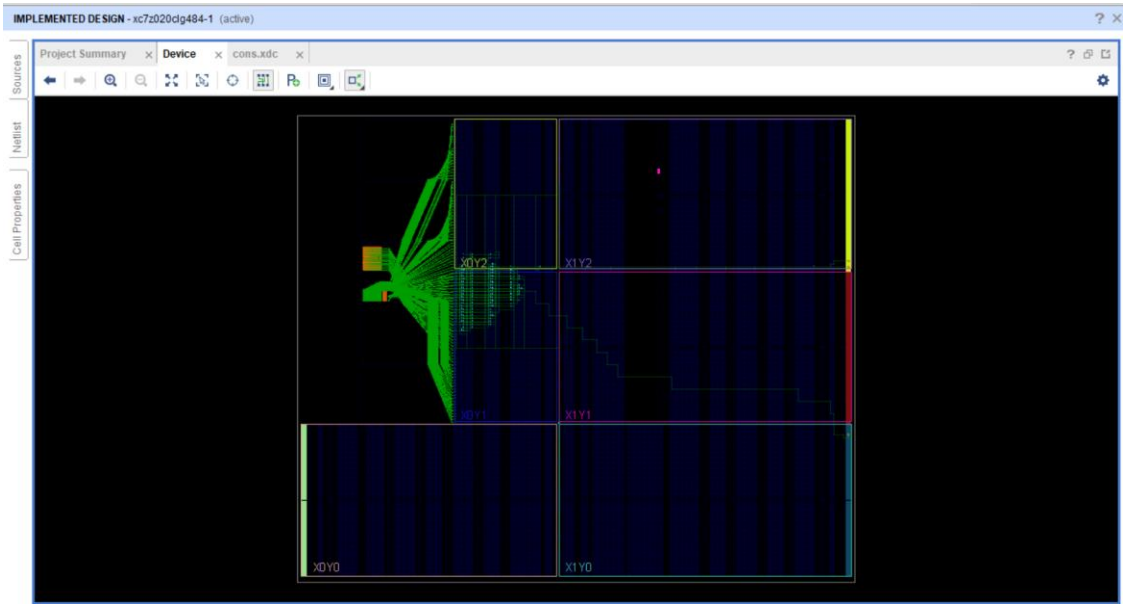

Figure 3 — Implemented design, device view. Small footprint, as expected for a 2-input AND gate plus two AXI GPIO controllers.

4. Software Design (Processing System)

A bare-metal C application (inv.c, from the Xilinx “Hello World” template) runs on ps7_cortex9_0, using the Xilinx GPIO driver (xgpio.h) to talk to the two AXI GPIO peripherals:

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xgpio.h"
#include "xparameters.h"

int main()
```

```
{
    init_platform();
    int a;
    int y;
    XGpio input, output;
    XGpio_Initialize(&input,  XPAR_AXI_GPIO_0_DEVICE_ID);
    XGpio_Initialize(&output, XPAR_AXI_GPIO_1_DEVICE_ID);
    XGpio_SetDataDirection(&input,  1, 1);   // channel 1 = input
    XGpio_SetDataDirection(&output, 1, 0);   // channel 1 = output

    while (1) {
        a = XGpio_DiscreteRead(&input, 1);   // read AND-gate output
        y = ~a;                              // software inversion (NAND)
        XGpio_DiscreteWrite(&output, 1, y);  // drive NAND result
    }

    cleanup_platform();
    return 0;
}
```

4.1 Code Walkthrough

- XGpio_Initialize binds each handle to its AXI GPIO device ID.
- XGpio_SetDataDirection sets axi_gpio_0 as input, axi_gpio_1 as output.
- In the while(1) loop: read the AND result, invert with ~a, write it out — a continuous software inverter.

Built as application project inv_logic against hardware platform nand_zynq_sys_wrapper_hw_platform_0:

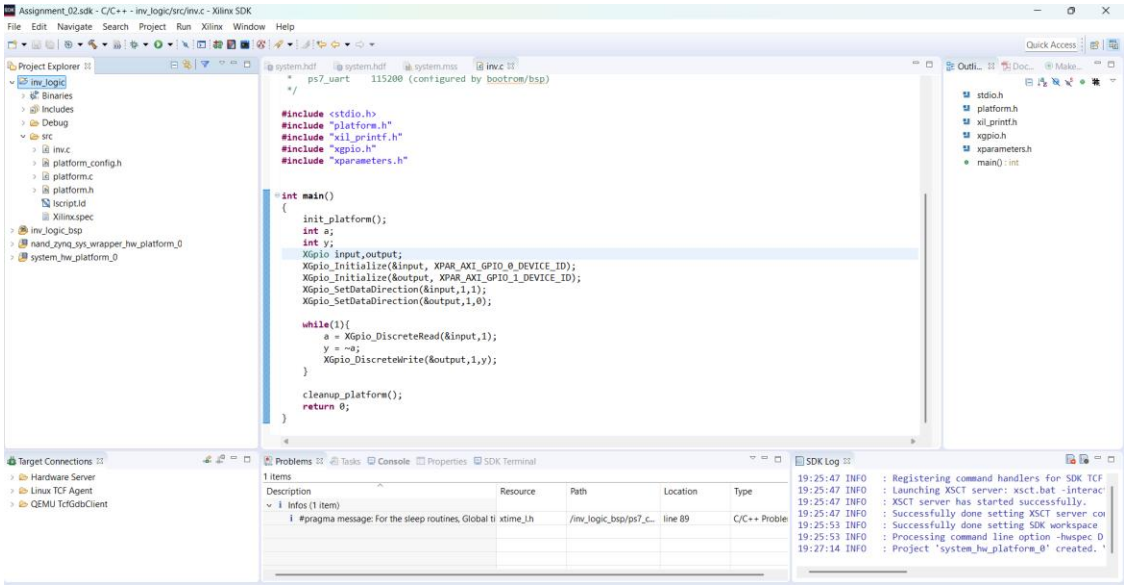

Figure 4 — inv.c in Xilinx SDK, project inv_logic, targeting ps7_cortex9_0.

5. Build Steps Completed

- Vivado project created for the ZedBoard; AND-logic RTL (nand_gate.v) added.
- Block design built: ZYNQ7 PS + Block Automation for DDR/FIXED_IO.
- AND-gate RTL and two AXI GPIOs (1-bit in / 1-bit out) added and wired; ports marked external; Connection Automation run.
- Design validated with no errors; HDL wrapper created and set as top.
- .xdc constraints written; bitstream generated; hardware exported with bitstream.
- inv_logic application created from the Hello World template, edited per Section 4, and built successfully.

6. Limitations

The project stops after the build step — programming the FPGA and running the ELF need a physical ZedBoard, which wasn't available. So the bitstream was never downloaded, and the NAND truth table was never verified on real hardware.

Everything up to that point — synthesis, implementation, bitstream generation, and C code build — completed without errors, giving good confidence the design is correct. Next step, once a board is available: Program Device, then Run on Hardware, then confirm LD0 follows the NAND truth table as SW0/SW1 toggle.